# Union–Find Data Structure

## Operations

- Make-Set(x) – Creates a disjoint set with x as its only member.
- Find-Set(x) – Returns the name of the set to which x belongs.
- Union(x,y) – Unites the disjoint sets to which x and y belong to a single set.

## *Heuristics*

- ***Union by rank*** – We keep a rank field that is an upper bound on the height of the node.
- ***Path compression*** – We compress each traversed path causing every node on the path to point directly to the root.

# Union–Find Data Structure

- Make-Set(x)

1. p[x]← x
2. rank[x] ← 0

- Union(x,y)

1. Link(Find-Set(x),Find-Set(y))

- Find-Set(x)

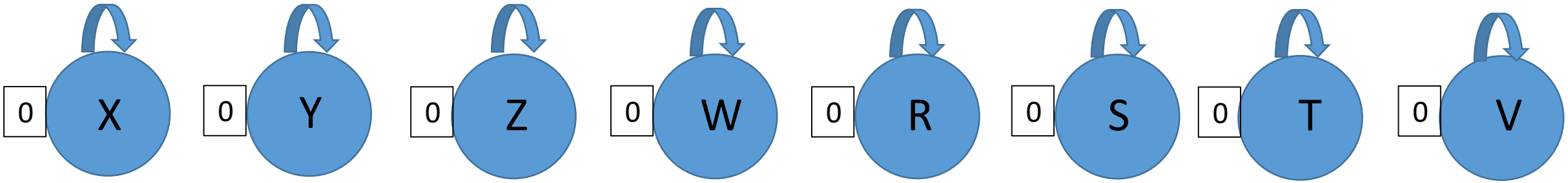
1. If x ≠ p[x]
2. then p[x]← Find-Set(p[x])
3. return p[x]

- Link(x,y)

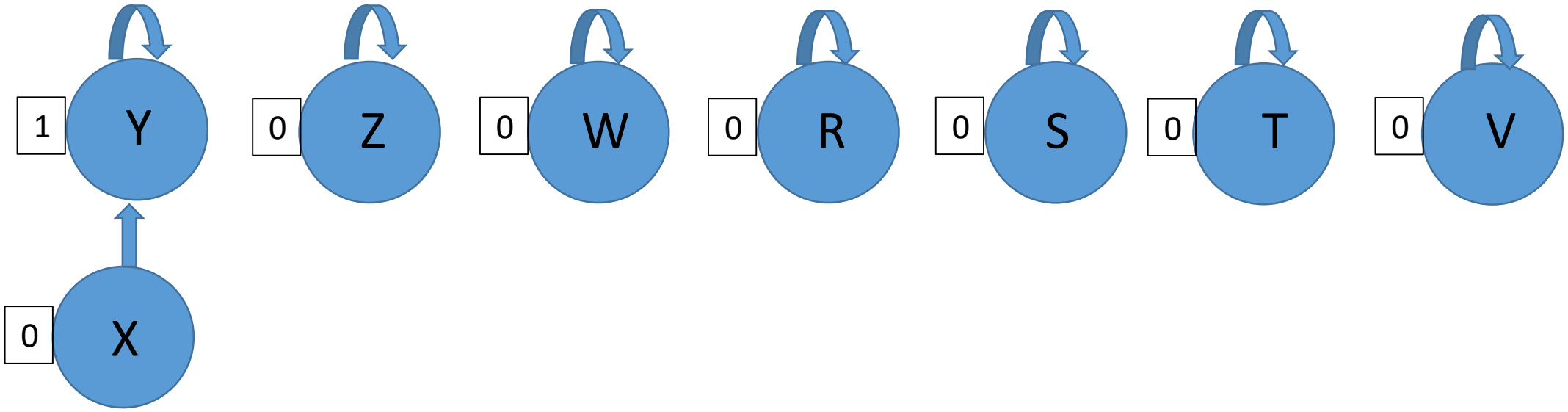
1. If rank[x]>rank[y]
2. then p[y] ← x
3. else p[x] ←y
4. if rank[x]=rank[y]
5. then rank[y] ← rank[y]+1

*m* Union-Find operations
with *n* Make-Set operations cost:
$O(m \cdot \alpha(m,n))$

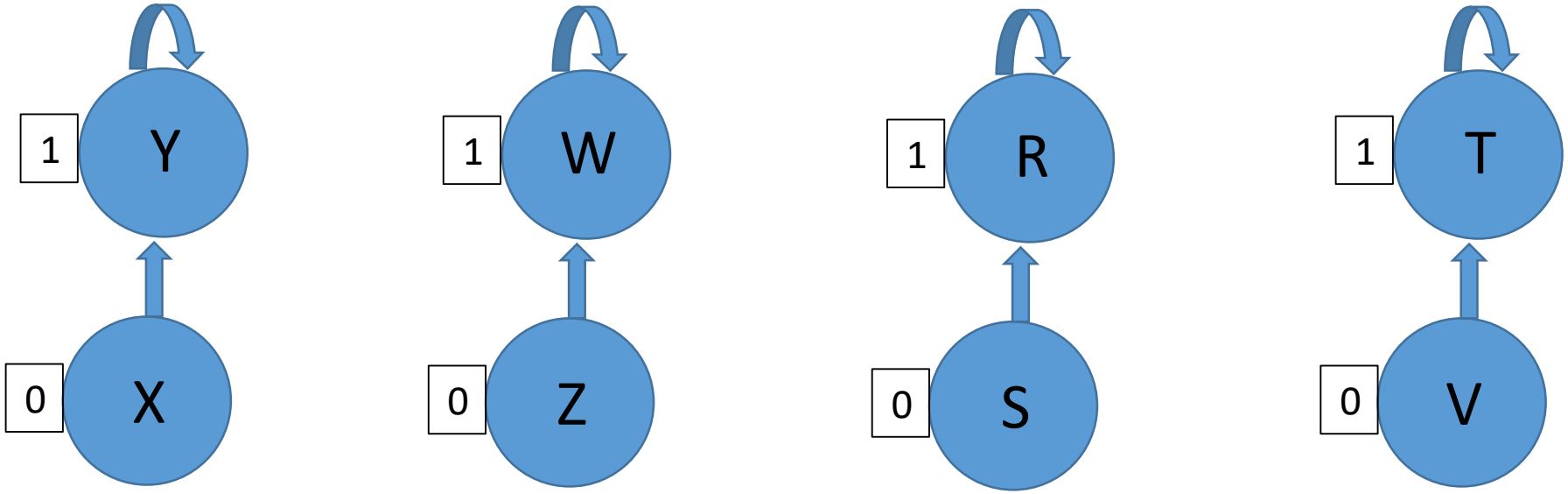
# Union–Find Data Structure



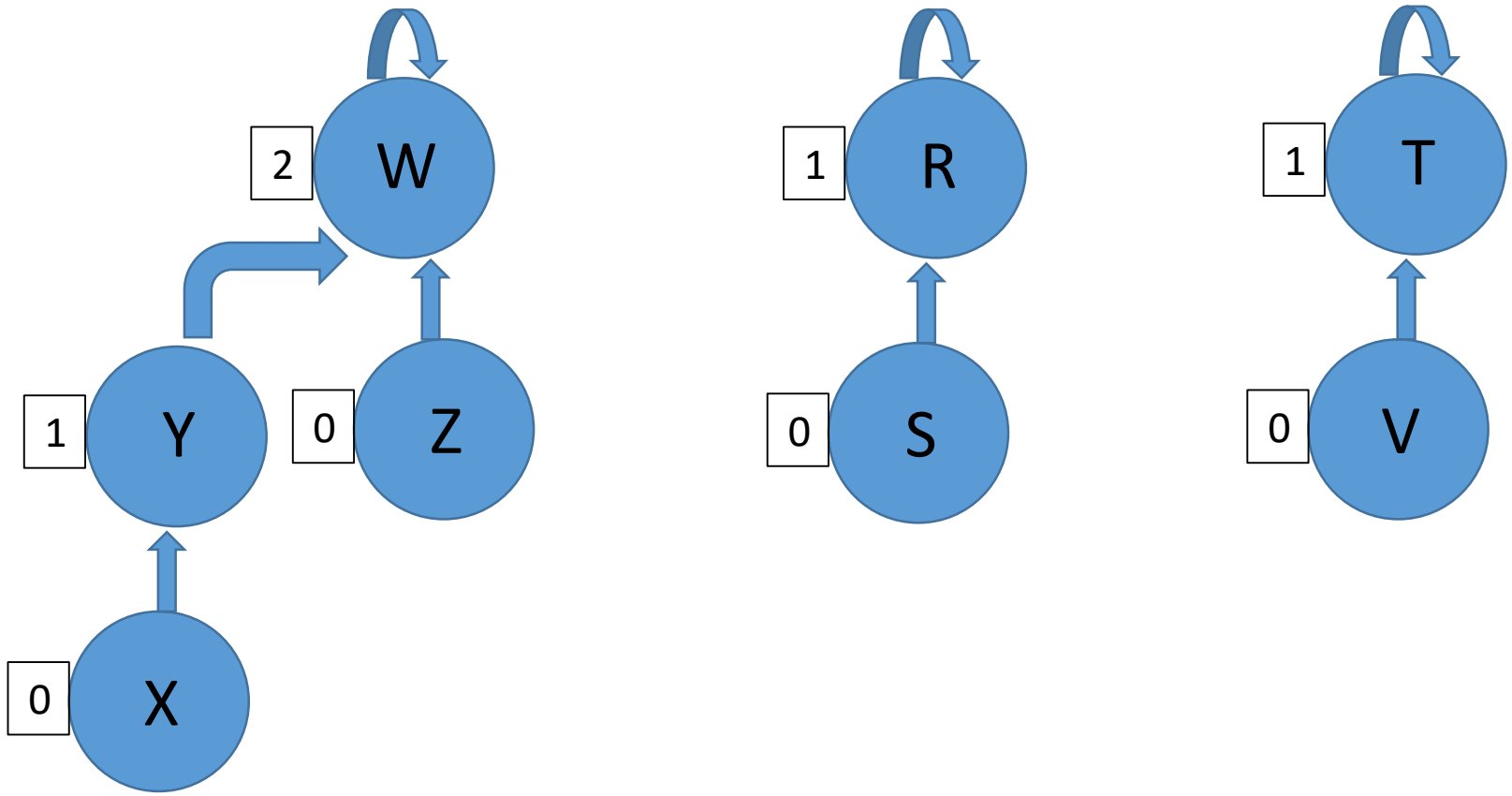
# Union–Find Data Structure



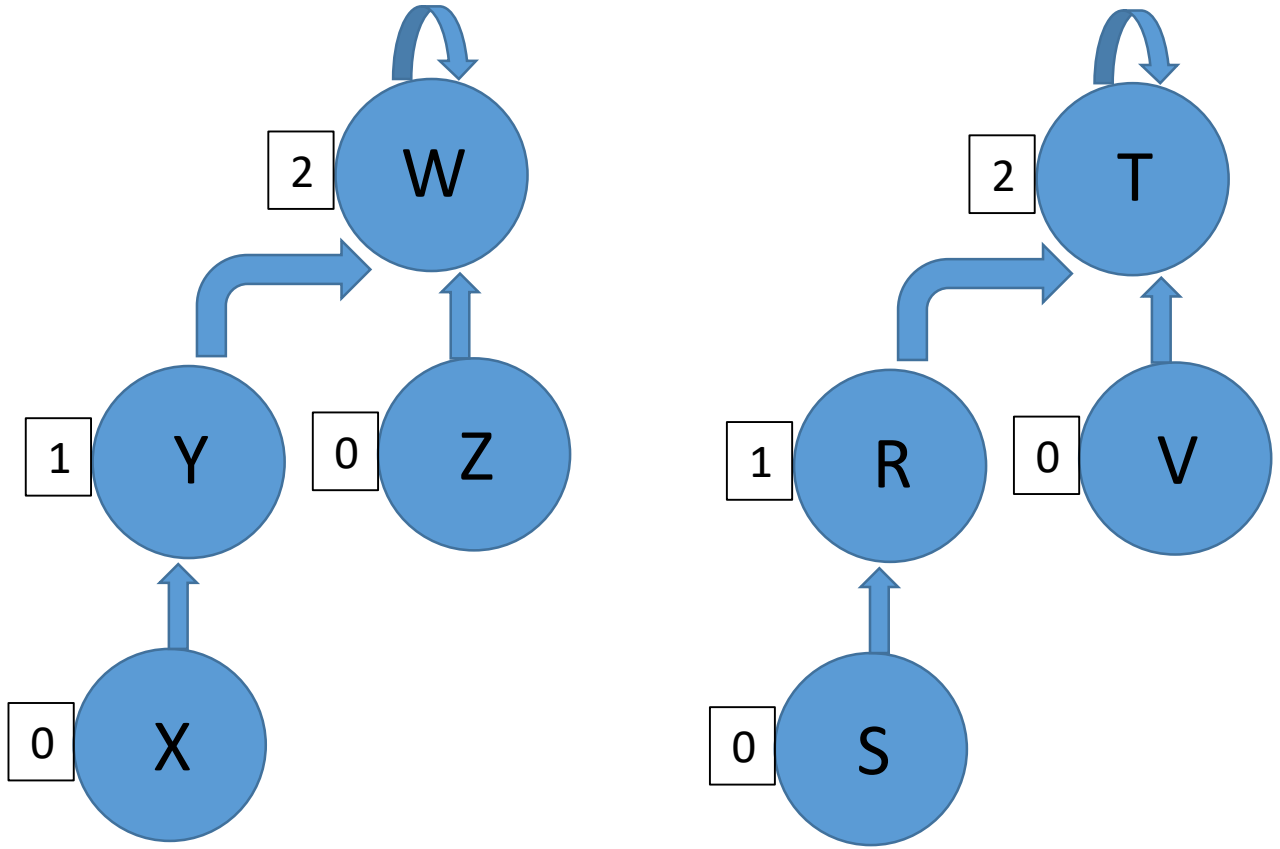
# Union–Find Data Structure



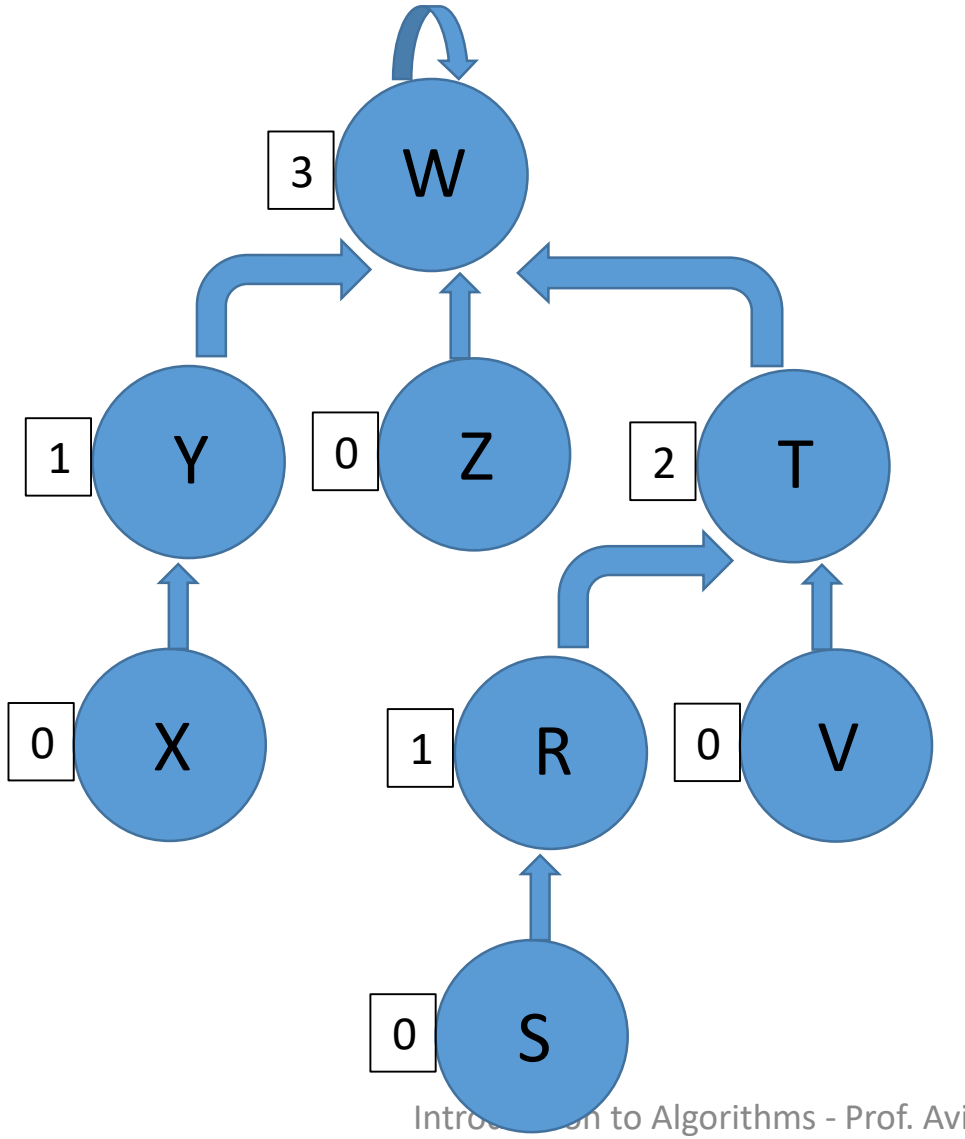
# Union–Find Data Structure



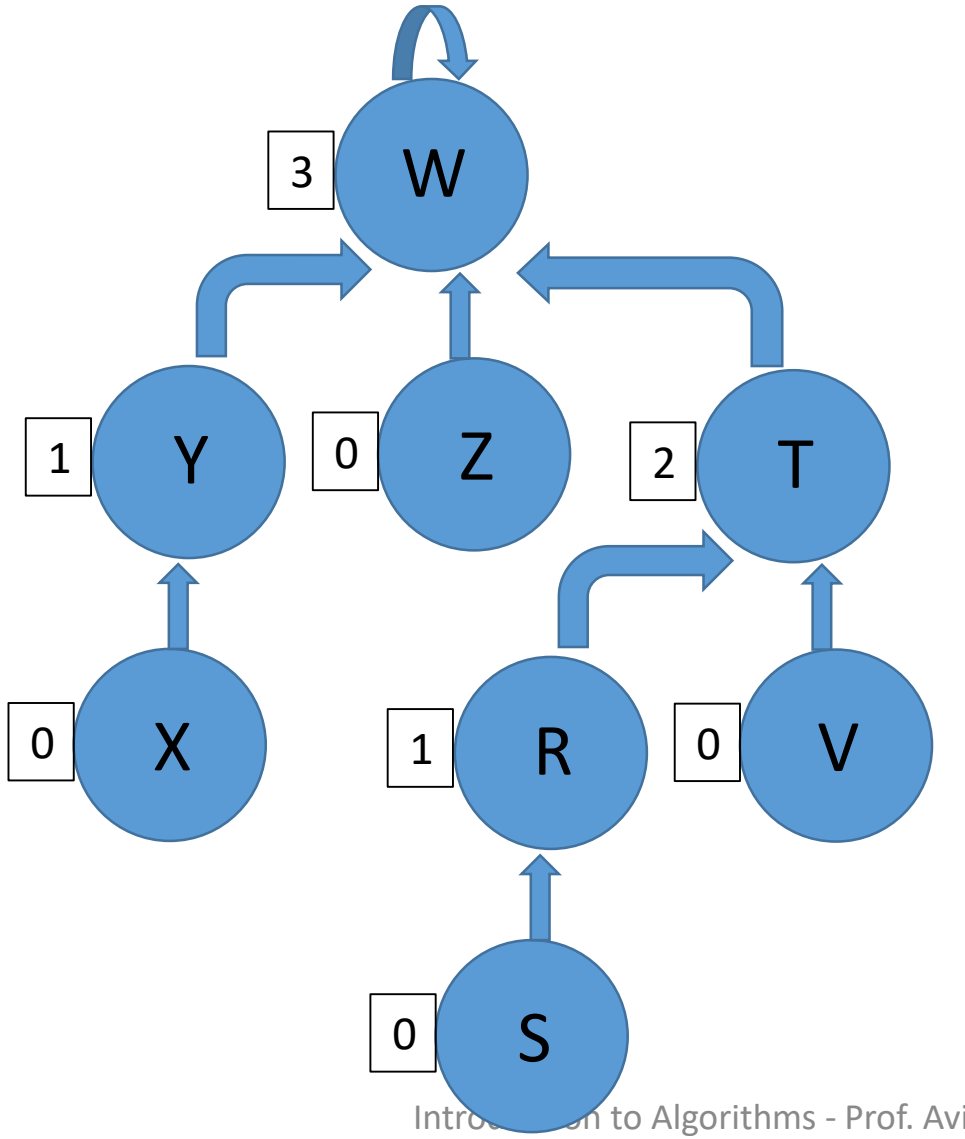
# Union–Find Data Structure



# Union–Find Data Structure

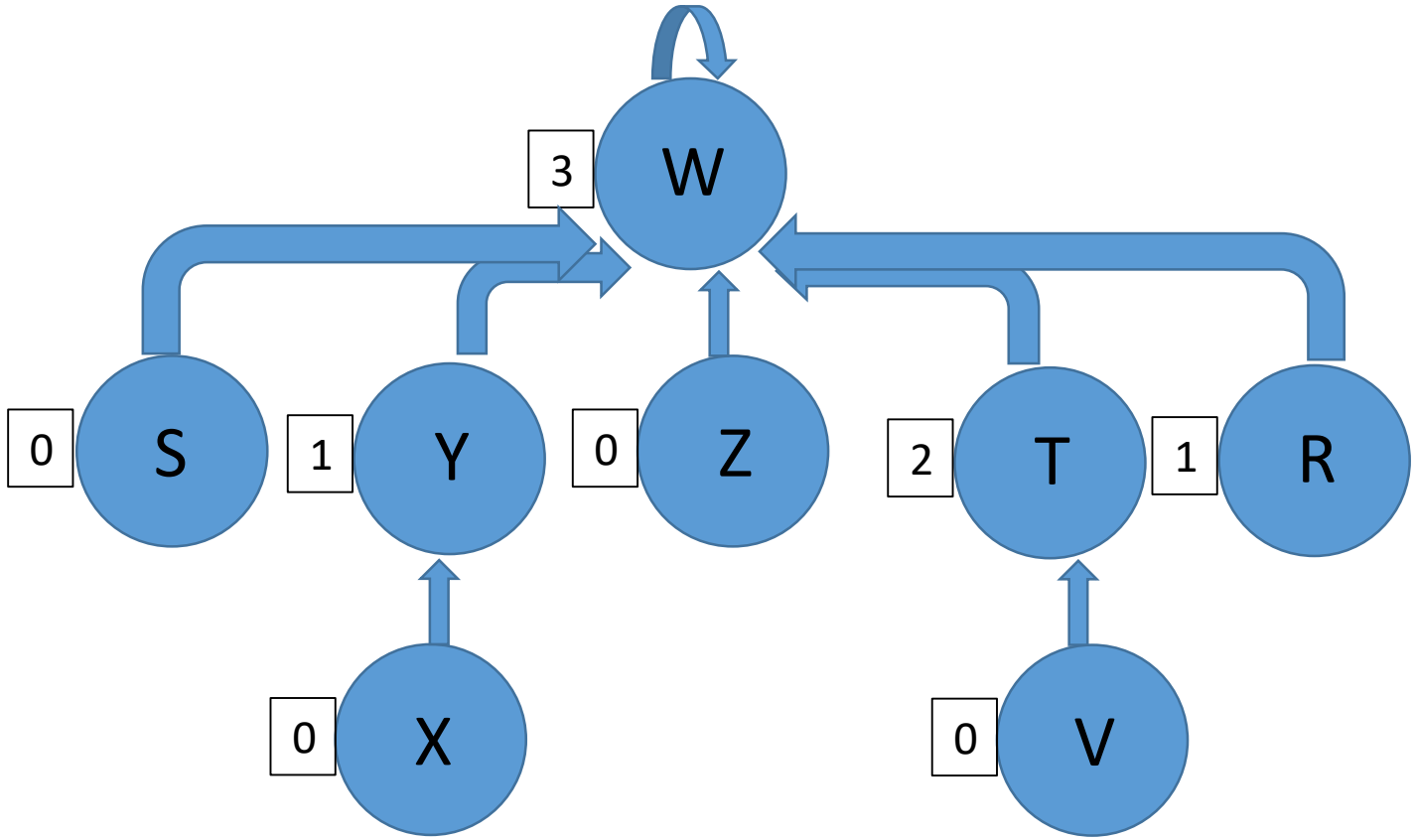


# Union–Find Data Structure




Find-Set(S)

# Union–Find Data Structure



# Union–Find Data Structure

**Claim 1:**

Let x be the root of a set in the union-find forest, then the number of nodes in the set of x is at least $2^{rank[x]}$.

**Proof:** By induction on $\mathrm{rank}[\mathrm{x}]$.

**Base case:** rank[x]=0. In this case, x is a single node set created by MAKE_SET function.

Therefore, we have that the size of its set is $1 = 2^0 = 2^{rank[x]}$, as required.

# Union–Find Data Structure

**Proof (cont.):**

**Induction step:** Assume by induction claim holds for all root nodes with rank=k. We show it holds for root nodes with rank=k+1.

# Union–Find Data Structure

**Proof (cont.):**

**Induction step:** Assume by induction claim holds for all root nodes with rank=k. We show it holds for root nodes with rank=k+1.

Any root node x with rank[x]=k+1 was a root node x with rank[x]=k, that was linked to another root node, say y, with rank[y]=k.

# Union–Find Data Structure

**Proof (cont.):**

**Induction step:** Assume by induction claim holds for all root nodes with rank=k. We show it holds for root nodes with rank=k+1.

Any root node x with rank[x]=k+1 was a root node x with rank[x]=k, that was linked to another root node, say y, with rank[y]=k (because line 5 in the LINK procedure is the only pseudo-code line increasing the rank field and reached only in this scenario). Thus, x's set-size is the size of both x's and y's sets.

# Union–Find Data Structure

**Proof (cont.):**

**Induction step:** Assume by induction claim holds for all root nodes with rank=k. We show it holds for root nodes with rank=k+1.

Any root node x with rank[x]=k+1 was a root node x with rank[x]=k, that was linked to another root node, say y, with rank[y]=k (because line 5 in the LINK procedure is the only pseudo-code line increasing the rank field and reached only in this scenario). Thus, x's set-size is the size of both x's and y's sets.

By induction hypothesis x's and y's sets sizes are at least $2^{rank[x]}$ and $2^{rank[y]}$, that is: $2^k$.

# Union–Find Data Structure

**Proof (cont.):**

**Induction step:** Assume by induction claim holds for all root nodes with rank=k. We show it holds for root nodes with rank=k+1.

Any root node x with rank[x]=k+1 was a root node x with rank[x]=k, that was linked to another root node, say y, with rank[y]=k (because line 5 in the LINK procedure is the only pseudo-code line increasing the rank field and reached only in this scenario). Thus, x's set-size is the size of both x's and y's sets.

By induction hypothesis x's and y's sets sizes are at least $2^{rank[x]}$ and $2^{rank[y]}$, that is: $2^k$.

Thus, root node x's set-size after the LINK operation is at least:

$$2^k + 2^k = 2 \cdot 2^k = 2^{k+1},$$

as required.

# Union–Find Data Structure

**Conclusion:**

The rank of any node x in the union-find forest is at most $\log n$, where n is the number of initial MAKE-SET operations.

**Proof:** First observe that the greatest rank value is reached by a root node in the union-find forest. By claim 1, for any root node x in the union-find forest the number of nodes in x's set is at least $2^{rank[x]}$.
However, the total number of nodes is n. Therefore,

$$2^{rank[x]} \leq n$$

Thus,

$$rank[x] \leq \log n$$

# Finding Minimum Spanning Tree

## Kruskal's Algorithm Cost

| Command Number | Cost |
|---|---|
| 1 | $\Theta(1)$ |
| 2-3 | $\Theta(\|V\|)$ |
| 4 | $\Theta(\|E\| \log\|E\|)$ |
| 5-8 | $\Theta\big(\|E\| \cdot \alpha(\|E\|, \|V\|)\big)$ |
| 9 | $\Theta(1)$ |
| Total | $\Theta(\|E\| \log\|V\|)$ |